

TUGAS PENDAHULUAN

MODUL 13

Mei sari mantiantini/SE0701/2311104012

- A. Berikan salah satu contoh kondisi dimana design pattern “Observer” dapat digunakan

Jawab :

Design pattern Observer sangat cocok digunakan saat terdapat satu objek (Subject) yang ingin memberi tahu banyak objek lain (Observer) ketika terjadi perubahan.

Contohnya:

Pada aplikasi cuaca (weather app), terdapat pusat data cuaca (Subject) yang akan menginformasikan perubahan suhu, kelembapan, atau tekanan udara kepada berbagai tampilan UI (Observer) seperti grafik, laporan teks, atau notifikasi.

- B. Berikan penjelasan singkat mengenai langkah-langkah dalam mengimplementasikan design pattern “Observer” .

Jawab :

- Buat interface Observer
- Buat interface Subject
- Buat kelas Subject konkret
- Buat kelas Observer konkret
- Hubungkan Observer ke Subject

- C. Berikan kelebihan dan kekurangan dari design pattern “Observer”

Jawab :

Kelebihan:

Loose coupling (ketergantungan rendah) antara Subject dan Observer, memudahkan sinkronisasi antara satu sumber data dan banyak konsumen., modular: mudah menambah atau menghapus observer.

Kekurangan:

Jika jumlah observer sangat banyak, bisa mempengaruhi kinerja, sulit di-debug jika notifikasi saling bertumpuk atau berantai, ada risiko memory leaks jika observer tidak dilepas dengan benar.

Program.cs

```
using System;
using System.Collections.Generic;

// Interface Observer
interface IObserver
{
    void Update(ISubject subject);
}

// Interface Subject
interface ISubject
{
    void Attach(IObserver observer);
    void Detach(IObserver observer);
    void Notify();
}

// Subject konkret
class Subject : ISubject
{
    public int State { get; set; } = 0;

    private List<IObserver> _observers = new List<IObserver>();

    public void Attach(IObserver observer)
    {
        Console.WriteLine("Subject: Attached an observer.");
        _observers.Add(observer);
    }

    public void Detach(IObserver observer)
    {

```

```

        _observers.Remove(observer);
        Console.WriteLine("Subject: Detached an observer.");
    }

    public void Notify()
    {
        Console.WriteLine("Subject: Notifying observers...");

        foreach (var observer in _observers)
        {
            observer.Update(this);
        }
    }

    public void SomeBusinessLogic()
    {
        Console.WriteLine("\nSubject: I'm doing something important.");
        this.State = new Random().Next(0, 10);

        Console.WriteLine("Subject: My state has just changed to: " + this.State);
        this.Notify();
    }
}

// Observer konkret A
class ConcreteObserverA : IObserver
{
    public void Update(ISubject subject)
    {
        if ((subject as Subject).State < 3)
        {
            Console.WriteLine("ConcreteObserverA: Reacted to the event.");
        }
    }
}

```

```

    }
}

// Observer konkret B
class ConcreteObserverB : IObserver
{
    public void Update(ISubject subject)
    {
        if ((subject as Subject).State == 0 || (subject as Subject).State >= 7)
        {
            Console.WriteLine("ConcreteObserverB: Reacted to the event.");
        }
    }
}

// Program utama
class program
{
    static void Main(string[] args)
    {
        var subject = new Subject();

        var observerA = new ConcreteObserverA();
        subject.Attach(observerA);

        var observerB = new ConcreteObserverB();
        subject.Attach(observerB);

        subject.SomeBusinessLogic();
        subject.SomeBusinessLogic();
    }
}

```

```

        var observerB = new ConcreteObserverB();
        subject.Attach(observerB);

        subject.SomeBusinessLogic();
        subject.SomeBusinessLogic();

        subject.Detach(observerB);

        subject.SomeBusinessLogic();
    }
}

```

Penjelasan :

```

static void Main(string[] args)
{
    var subject = new Subject();

```

Membuat objek subject dari kelas Subject, subject ini akan menjadi **publisher** atau objek yang akan menginformasikan perubahan ke observer.

```
var observerA = new ConcreteObserverA();  
subject.Attach(observerA);
```

Membuat objek observerA dari kelas ConcreteObserverA, Ini adalah salah satu **observer** yang akan menerima notifikasi dari subject.

Output :

```
PS D:\SEMESTER 4_Praktikum KPL\KPL_MEISARIMANTIANI\2311104012_SE0701\13_Design_Pattern\tpmodu13_2311104012> dotnet run  
Subject: Attached an observer.  
Subject: Attached an observer.  
  
Subject: I'm doing something important.  
Subject: My state has just changed to: 4  
Subject: Notifying observers...  
  
Subject: I'm doing something important.  
Subject: My state has just changed to: 8  
Subject: Notifying observers...  
ConcreteObserverB: Reacted to the event.  
Subject: Detached an observer.  
  
Subject: I'm doing something important.  
Subject: My state has just changed to: 9  
Subject: Notifying observers...
```